

Aula: Primeiros Passos com Node.js

Crie do zero uma lista de tarefas funcional. Você vai entender o papel do backend, do frontend e como os dois conversam por HTTP — sem instalar nada além do Node.js, sem frameworks.

-  Node.js
-  HTML + JS
-  Rotas HTTP
-  JSON como banco
-  Sem frameworks

ÍNDICE DA AULA

- 1. Conceitos básicos
- 2. Criando o projeto
- 3. Rodando pela 1ª vez
- 4. Usando a lista
- 5. O caminho do clique
- 6. Glossário
- 7. Quiz de fixação
- 8. Próximos passos

1. Conceitos básicos

O que é Node.js?

Node.js é um programa que executa JavaScript **fora do navegador**. Antes dele, o JavaScript só rodava dentro de páginas web. Com Node, você pode usar JavaScript para:

- Criar servidores
- Ler e escrever arquivos no computador
- Criar APIs, bots, ferramentas de linha de comando

Frontend vs Backend

	Frontend	Backend
Onde roda	No navegador do usuário	No computador/servidor
O que faz	Mostra a tela, botões, formulários	Guarda dados, aplica regras, responde pedidos
Tecnologias	HTML, CSS, JavaScript	Node.js (nesta aula)

 **Na nossa aula vamos construir os dois:** um frontend (a página HTML que você vê) e um backend (o Node.js que guarda as tarefas num arquivo). Eles conversam por **HTTP** — o mesmo protocolo que seu

navegador usa para abrir qualquer site.

2. Criando o projeto do zero

1 Criar a pasta do projeto

Abra o terminal e rode:

Terminal (bash)

[Copiar](#)

```
mkdir todo-local
cd todo-local
```

Comando

O que faz

`mkdir todo-local`

Cria uma pasta chamada `todo-local`

`cd todo-local`

Entra nessa pasta (todo comando seguinte roda dentro dela)

`todo-local/` ← *pasta vazia, recém-criada*

2 Criar a pasta `public/`

Terminal

[Copiar](#)

```
mkdir public
```

💡 **Por que criar essa pasta?** Por convenção, arquivos que o navegador vai acessar (HTML, CSS, imagens) ficam dentro de uma pasta chamada `public`. Isso separa claramente **o que o usuário vê** (`public/`) do **que o servidor faz** (fora de `public/`).

`todo-local/`
└─ `public/` ← *vai receber o HTML*

3 Criar o arquivo `tarefas.json`

Dentro de `todo-local/`, crie um arquivo chamado `tarefas.json` com este conteúdo:

 **todo-local/tarefas.json** Copiar

```
[ ]
```

 **O que é esse arquivo?** É o nosso "banco de dados". Um arquivo de texto simples onde as tarefas ficam guardadas entre uma sessão e outra. Os `[ ]` significam "lista vazia". **JSON** é um formato de texto para representar dados — fácil de ler tanto para humanos quanto para o Node.js.

```
todo-local/  
├─ public/  
└─ tarefas.json ← banco de dados (começa vazio)
```

4 Criar o arquivo `public/index.html`

Este arquivo é a **interface visual**: HTML (estrutura), CSS (aparência) e JavaScript (lógica que conversa com o backend), tudo junto.

 **public/index.html** Copiar

```
<!DOCTYPE html>  
<html lang="pt-br">  
<head>  
  <meta charset="UTF-8">  
  <title>Lista de Tarefas</title>  
  <style>  
    body { font-family: sans-serif; max-width: 500px; margin: 40px auto; }  
    input { padding: 8px; width: 70%; }  
    button { padding: 8px 12px; cursor: pointer; }  
    li { margin: 8px 0; display: flex; justify-content: space-between; align-items: center; }  
    .feita { text-decoration: line-through; color: gray; }  
  </style>  
</head>  
<body>  
  <h1>📌 Minhas Tarefas</h1>  
  
  <input id="campo" placeholder="Digite uma tarefa..." />  
  <button onclick="adicionar()">Adicionar</button>  
  
  <ul id="lista"></ul>  
  
<script>
```

```
todo-local/  
├─ public/
```

- | — `index.html` ← *interface visual (frontend)*
- | — `tarefas.json`

5 Criar o arquivo `server.js`

Este é o **backend** — o **cérebro da aplicação**. Ele escuta pedidos do navegador (HTTP), lê e escreve no arquivo `tarefas.json` e devolve respostas.

`todo-local/server.js`

 Copiar

```
// Rota 1: entregar a página HTML ao navegador
if (req.method === 'GET' && req.url === '/') {
  const html = fs.readFileSync(path.join(__dirname, 'public', 'index.html'));
  res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
  res.end(html);
  return;
}

// Rota 2: devolver a lista de tarefas (GET /tarefas)
if (req.method === 'GET' && req.url === '/tarefas') {
  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify(lerTarefas()));
  return;
}

// Rota 3: adicionar tarefa (POST /tarefas)
if (req.method === 'POST' && req.url === '/tarefas') {
  let corpo = '';
  req.on('data', pedaco => corpo += pedaco);
  req.on('end', () => {
    const { titulo } = JSON.parse(corpo);
    const tarefas = lerTarefas();
    // ...
  });
}
```

Estrutura final:

```
todo-local/
├─ public/
├─ index.html ← frontend (o que o usuário vê)
├─ server.js ← backend (o cérebro Node.js)
└─ tarefas.json ← banco de dados (persistência)
```

3. Rodando pela primeira vez

No terminal, dentro da pasta `todo-local`, rode:

Terminal Copiar

```
node server.js
```

O que acontece:

1. O Node.js lê e executa o arquivo `server.js`
2. Ele abre a porta 3000 do seu computador e fica "escutando"
3. Aparece no terminal:  **Servidor rodando! Abra: <http://localhost:3000>**

 **Importante:** o terminal vai ficar "travado" — isso é normal. O servidor está rodando. Para pará-lo, aperte **Ctrl + C**.

Agora abra o navegador em: <http://localhost:3000>

 4. Usando a lista

Tente isto e observe o que acontece:

1. Digite **Comprar pão** → clique **Adicionar** → aparece na lista.
2. Clique no texto da tarefa → ela fica riscada (feita).
3. Clique no  → ela é apagada.
4. Abra o arquivo `tarefas.json` no editor → veja as tarefas salvas em JSON.
5. Feche o navegador, abra de novo, acesse a URL → as tarefas continuam lá.

 **O item 5 é a prova da persistência:** os dados não vivem só na tela, eles ficam num arquivo do seu computador. Essa é a diferença entre um site comum e uma aplicação com backend.

 5. O que acontece quando você clica em "Adicionar"

Este é o coração da aula. Acompanhe o caminho da informação:



Você digita "Comprar pão" e clica em Adicionar

Usuário interage com o navegador.



index.html (frontend, no navegador)

Dispara `fetch('/tarefas', { method: 'POST', body: {...} })`.

viaja pela rede local, protocolo HTTP



server.js (backend, no Node.js)

- recebe a requisição `POST /tarefas`
- lê o arquivo `tarefas.json` com `fs.readFileSync`
- adiciona a nova tarefa na lista
- salva o arquivo `tarefas.json` com `fs.writeFileSync`

responde "ok" para o navegador



index.html chama carregar()

Faz um novo `fetch GET /tarefas` para pegar a lista atualizada.



server.js responde com a lista atualizada

Devolve o JSON com todas as tarefas atuais.



Você vê a nova tarefa aparecer na tela

O ciclo se completa em milissegundos.



Resumo em uma frase: o frontend pede, o backend processa e salva, o frontend atualiza a tela.



6. Glossário de tudo que apareceu

Pastas e arquivos

Nome	Tipo	Para que serve
<code>todo-local/</code>	pasta	Raiz do projeto. Tudo fica dentro dela.
<code>public/</code>	pasta	Arquivos que o navegador pode acessar (HTML, CSS, imagens).
<code>public/index.html</code>	arquivo	A interface da aplicação (frontend).
<code>server.js</code>	arquivo	O servidor Node.js (backend).
<code>tarefas.json</code>	arquivo	Onde as tarefas ficam salvas (persistência).

Conceitos

Termo	Significado
Node.js	Programa que executa JavaScript fora do navegador.

Termo	Significado
Frontend	O que o usuário vê (roda no navegador).
Backend	O que processa e guarda dados (roda no Node.js).
HTTP	Protocolo de comunicação entre frontend e backend.
Rota	Um "endereço" que o servidor reconhece (ex.: <code>/tarefas</code>).
GET / POST / PUT / DELETE	Verbos HTTP: ler, criar, atualizar, apagar.
JSON	Formato de texto para guardar/transportar dados.
require('http')	Importa um módulo que já vem embutido no Node.
fs	Módulo do Node para ler/escrever arquivos.
path	Módulo do Node para montar caminhos de arquivos.
localhost:3000	Endereço local do seu servidor na porta 3000.
fetch()	Função do navegador para fazer pedidos HTTP.
Persistência	Dados que sobrevivem ao fechar o programa.

Comandos usados

Comando	O que faz
<code>mkdir todo-local</code>	Cria a pasta do projeto
<code>cd todo-local</code>	Entra na pasta
<code>mkdir public</code>	Cria a pasta pública
<code>node server.js</code>	Executa o servidor
<code>Ctrl + C</code>	Para o servidor



7. Quiz de fixação (10 questões)

Marque as respostas e clique em **Verificar** no final.

1 O que é o Node.js?

☐ Um navegador de internet alternativo

☒ Um programa que executa JavaScript fora do navegador

☐ Uma linguagem de programação nova, diferente do JavaScript

☐ Um banco de dados relacional

2 Neste projeto, o arquivo `index.html` pertence ao frontend ou ao backend?

☒ Frontend, porque roda no navegador e mostra a tela ao usuário

☐ Backend, porque é lido pelo Node.js

☐ Ambos, com o mesmo peso

☐ Nenhum dos dois — é só um arquivo de configuração

3 Qual é o papel do arquivo `tarefas.json` ?

☐ Definir a aparência da página

☒ Servir como banco de dados simples que guarda as tarefas em texto

☐ Criar o servidor HTTP

☐ Compilar o JavaScript em código nativo

4 Para que serve a pasta `public/` ?

☐ Guardar o arquivo `server.js`

☒ Conter arquivos que o navegador vai acessar (HTML, CSS, imagens)

☐ Substituir o `tarefas.json`

☐ Ser o banco de dados relacional do projeto

5 Qual verbo HTTP é usado para **criar** uma nova tarefa?

☐ GET

☒ POST☐ PUT☐ DELETE

6 Por que as tarefas continuam lá depois de fechar o navegador?

☐ Porque o navegador guarda tudo automaticamente☒ Porque o backend salva os dados no arquivo `tarefas.json` em disco☐ Porque a página mantém os dados na memória RAM☐ Porque o HTML tem persistência embutida

7 Qual módulo do Node é usado para ler e escrever arquivos?

☐ `http`☐ `path`☒ `fs`☐ `json`

8 Se `node server.js` estiver rodando e você apertar `Ctrl + C` no terminal, o que acontece?

☐ O servidor continua rodando em segundo plano☒ O servidor é encerrado e o site para de responder em `localhost:3000`☐ O arquivo `tarefas.json` é apagado☐ O navegador fecha sozinho

9 O mesmo JavaScript pode rodar no navegador (frontend) e no Node.js (backend).

☒ Verdadeiro☐ Falso

10 Para rodar este projeto é necessário instalar frameworks como Express, React ou Angular.

☒ Verdadeiro☐ Falso[✓ Verificar Respostas](#)[🗑 Limpar Respostas](#) Resultado

10 / 10

Aproveitamento: 100%

🚀 8. Próximos passos

Quando você dominar o que vimos aqui, dá para evoluir:

- Instalar **Express** para escrever rotas de forma mais enxuta
- Adicionar **npm init** e entender o **package.json**
- Separar o CSS e o JS do HTML (pastas **css/** e **js/**)
- Trocar o **tarefas.json** por um banco real (SQLite, MongoDB)

✓ Mas nada disso antes de o básico acima estar sólido. Você já entendeu o ciclo completo: **frontend pede, backend processa e salva, frontend atualiza a tela.**